

Getting started with Python

This page tries to help you take your first steps with the Python programming language. In particular, it guides you in the first installation and through the configuration of your system so that you can follow and participate in the lab activities. The guide considers a Windows 10 system since it is the most common to find among students. However, many of the steps are very similar to those for other operating systems, like MacOS or Linux.

Once you got acquainted with the very basic, do not forget to install our suggested libraries and utilities.

Install the software

Typically this step should be necessary on a private machine, like your laptop or home personal computer. The PC we will use in labs should be already configured using a Windows 10 virtual machine.

What software do we use?

- Python 3 - in the version installed with the Anaconda manager that includes Python 3.7 (or newer)
- A text editor — NOT a document editor like Word — to write our programs in Python. We will consider two different alternatives: IDLE (the editor included in the standard Python installation) and Spyder (the editor included in Anaconda). However, any text editor is fine (notepad, notepad++, gedit, atom, pycharm and similar).

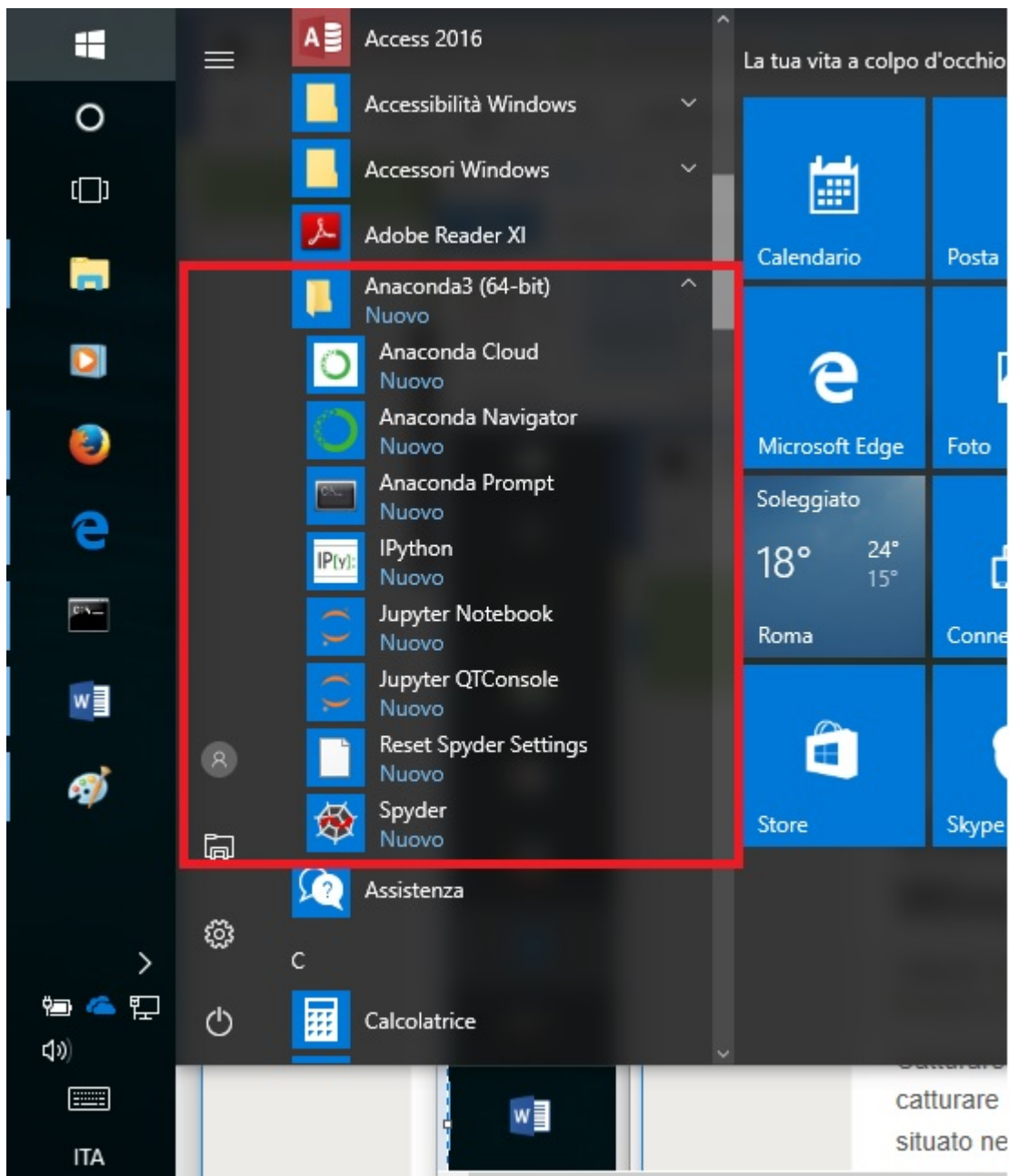
Download the software and install

Depending on the version of our PC (32-bit or 64-bit) and our operating system (Windows, MacOS, Linux), we download the right package from the [Anaconda download page](#) and install it following the instructions provided. Hopefully, everything should go straight, without any hassle.

Using the terminal and the editor

As mentioned, we will use the IDLE editor and the Spyder editor. Indeed, Spyder is much more complex software than IDLE, since — as we will see — it includes many additional features than a simple text editor (like debugging, integrated help and console, performance estimator, and so on). However, before using them, we will introduce the concept of *computer terminal*.

From the Windows menu, look for the Anaconda folder and select Anaconda prompt.



This will open a black window in which we can insert text.

```
(C:\Users\Angelo\Anaconda3) C:\Users\Angelo>
```

This window is called **terminal** and shows the **command line** (or prompt).

Digression: the name "terminal" derives from the fact that this software performs the same function as the old physical terminals, i.e., machines of the early computer age, whose only purpose was to act as a link between the human operator and the main-frame, receiving commands (the input) from the operator to the mainframe, and getting the result of the command (the output) back from the mainframe to the operator. In our context, we call "terminal" the interface we use to interact with our operating system. With that, we can make use of the same functionalities (and more) as those provided by the graphical interface – for example, starting applications, copying and handling files, changing the configuration of our system, and so on.

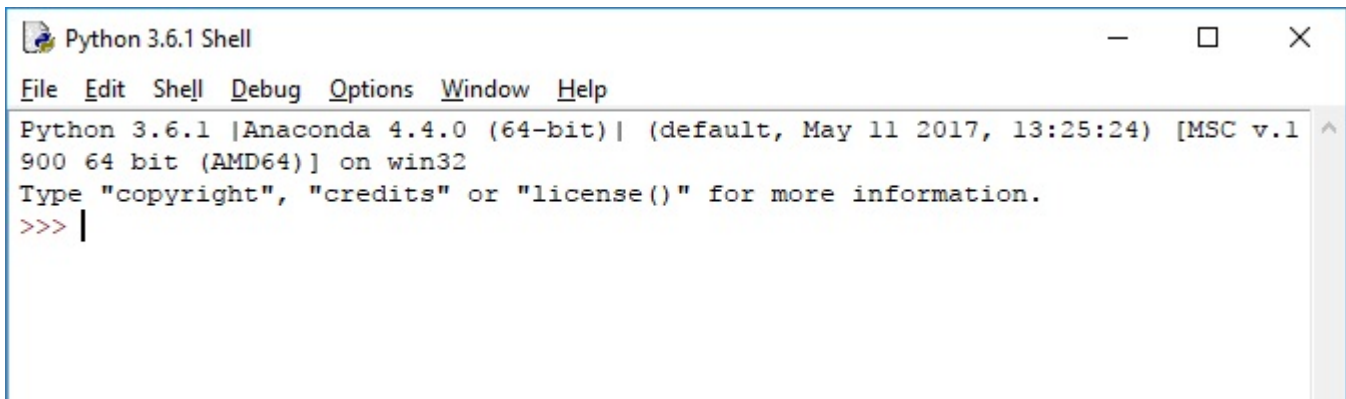
The command line includes a flashing cursor after a line similar to

```
C:\Users\Angelo>
```

It indicates that the system is ready to receive our commands. This line is called the **prompt** of the command line and usually shows the **current directory**. The current directory is crucial as it determines how we can indicate which programs to run. To run the IDLE editor, we type the word `idle` followed by the Enter key and wait.

```
(C:\Users\Angelo\Anaconda3\>) C:\Users\Angelo>idle
```

The above command asks the operating system to start the program under the name "idle.exe". A white window will open with a different prompt from the terminal one.

A screenshot of a 'Python 3.6.1 Shell' window. The title bar shows the window name and standard minimize, maximize, and close buttons. The menu bar includes 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The main text area displays the following information: 'Python 3.6.1 [Anaconda 4.4.0 (64-bit)] (default, May 11 2017, 13:25:24) [MSC v.1900 64 bit (AMD64)] on win32'. Below this, it says 'Type "copyright", "credits" or "license()" for more information.' and the prompt '>>>' is followed by a cursor.

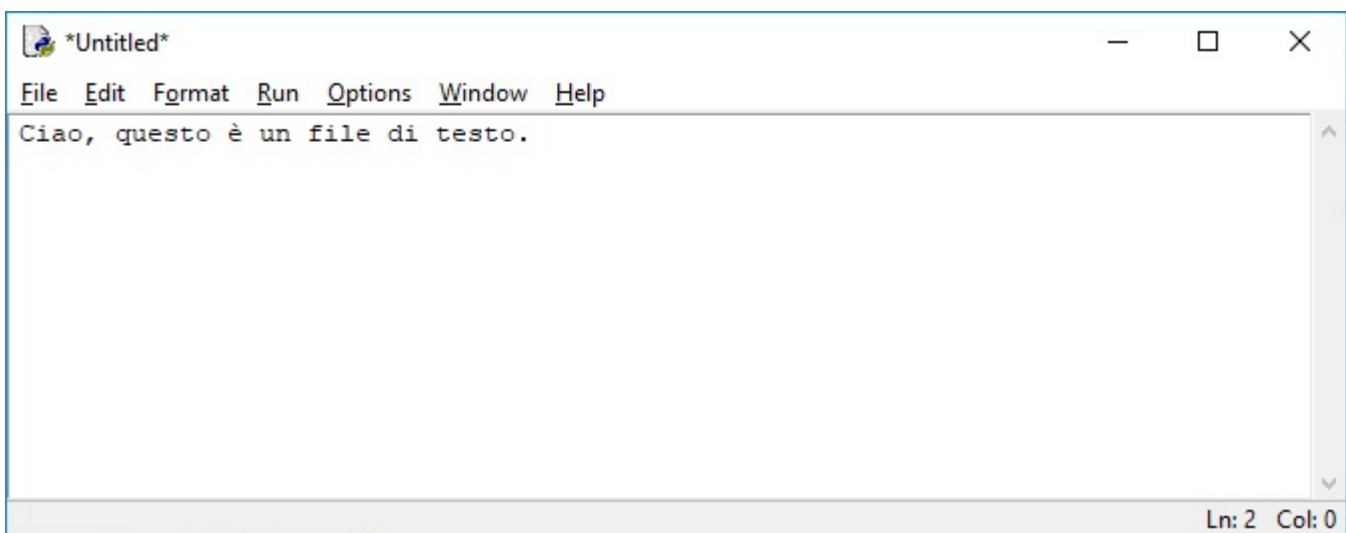
```
Python 3.6.1 Shell
File Edit Shell Debug Options Window Help
Python 3.6.1 [Anaconda 4.4.0 (64-bit)] (default, May 11 2017, 13:25:24) [MSC v.1
900 64 bit (AMD64)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> |
```

*BEWARE!! If the idle invocation returns a "Command not found" error, we have not executed the Anaconda command line. The error appears because the programs we installed with Anaconda are **not** known from the Windows command line, and we must manually include them: they are included in the Anaconda prompt. More precisely, the path in the filesystem where the executable files are located is not included in the PATH environment variable, which contains the list of paths used by the command line to search for executable files.*

This command line is now our interface with the Python interpreter! Here we can enter our Python commands and see the result of their execution.

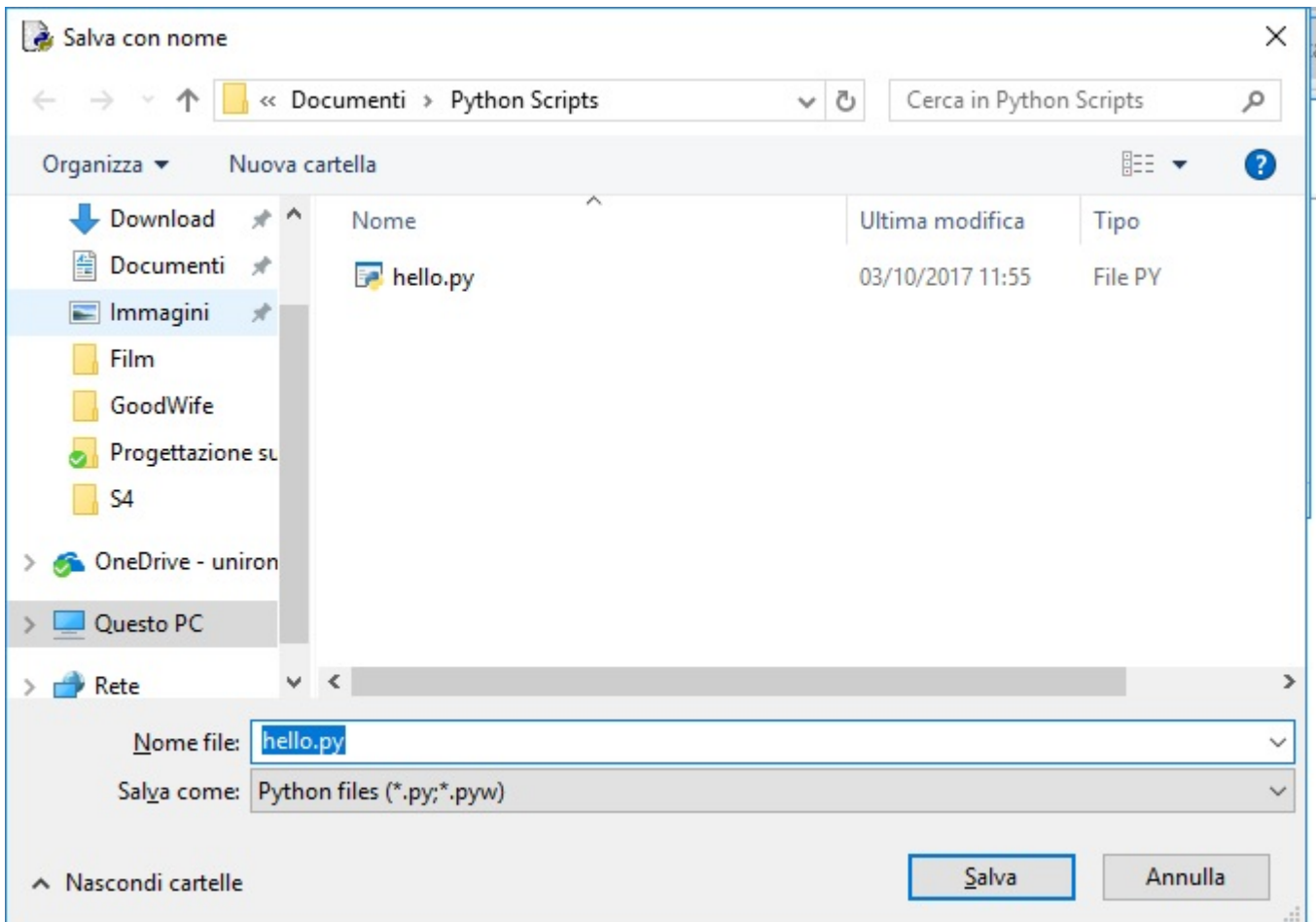
To open the editor, select from the **File** → **New File** menu entry to open the text editor. The text editor is different from a document editor (word processor): Microsoft Word or Google Docs or Apple Pages and the like can produce and format beautiful documents, but they are not designed to show and access the *actual* content files. When we create a "document", we use a lot of characters that, in reality, are not shown on the screen: the so-called special characters and formatting information. Our Python interpreter cannot run programs that include such characters: Python (and in general any programming language) needs files with pure text (**raw text**), i.e., sequences of characters. Text editors are in fact able to generate such file types.

Let us insert some text in the editor window...

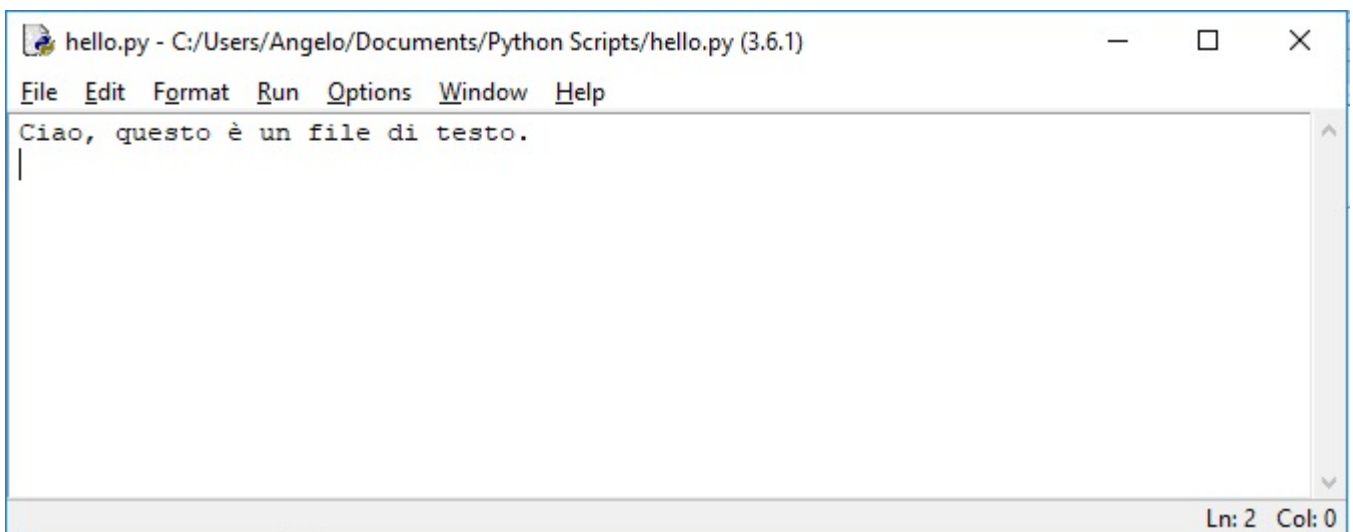
A screenshot of an 'Untitled*' text editor window. The title bar shows the window name and standard minimize, maximize, and close buttons. The menu bar includes 'File', 'Edit', 'Format', 'Run', 'Options', 'Window', and 'Help'. The main text area contains the text 'Ciao, questo è un file di testo.' in a monospaced font. The status bar at the bottom right shows 'Ln: 2 Col: 0'.

```
*Untitled*
File Edit Format Run Options Window Help
Ciao, questo è un file di testo.
Ln: 2 Col: 0
```

and save our new file: **File** → **Save As....** This command will let a new window pop up that allows us to choose where to save our file. Let us select **Documents** and then the **Python Scripts** directory. Thereafter, we choose a name for the file (for example **hello.py**).



At this stage, we can observe that the title of the editor window has changed and it shows the full path to our saved file (*C:/Users/Angelo/Documents/Python Scripts/hello.py*).



We should always take care of the file paths (i.e. the sequence of directories in which they are saved) because the command line needs to know where the files we want to use as input are, as we will see later. In the figure above, for example, the path to the *hello.py* file is *C:/Users/Angelo/Documents/Python Scripts/hello.py*, as indicated in the window title.

To sum up, we should have opened three windows:

- The terminal with the command line prompt (black screen)
- The python interpreter prompt (white screen)

- The text editor.

NOTE: Using IDLE as an editor is only one of the possible choices we have. We can use any text editor that can highlight Python syntax and save files in a format that Python can interpret.

How to use the command line

As we said, anything (and probably more) we can do with the graphical interface, we can do with the terminal. For a good understanding and proficient use of Python, it is very useful to know the command line a bit better.

One of the first things to know is the concept of **current directory** (or even **working directory**), which is often displayed directly in the command line prompt. For example, our command line in Windows is:

```
C:\Users\Angelo\Documents\Python Scripts>
```

meaning that the current directory is the "*Python Scripts*" directory, within the "*Documents*" directory, within the "*Angelo*" directory, within the "*Users*" directory of the "C:" disk.

NOTE: in Linux and MacOS, the prompt is typically different because the disk is not shown and the directory is relative to the user's home directory. So, to uniquely know the working directory, you must use the command

```
pwd
```

(print working directory). So, assuming our user is called *angelo*:

```
mypc:Documents/Python Scripts angelo$ pwd
/home/angelo/Documents/Python Scripts
mypc:Documents/Python Scripts angelo$
```

A handy tip about the command line is that it provides many keyboard shortcuts that make it quick to use. In particular:

- the **auto-completion** with the TAB key (the one just above the Caps Lock key of your keyboard), which helps you to automatically complete file names and commands. For example, if you start writing "cd Doc" and press the TAB key, the command line will automatically complete the command in "cd Documents"). Imagine the convenience if the file name we want to write is something like `user_assignments_settings_with_fixed_names_v.1.4.05-rev3.12.json` or alike...
- the **history navigation**, which consists of caching all the executed commands, and allowing you to recall any previously entered command to be executed again or altered before execution

To properly start using the command line, two more commands are fundamental:

- **dir** (or **ls** in linux/MacOS), which lists all the files in the current directory
- **cd**, which stands for *change directory*, which serves to change the current directory.

So, whenever we want to check that the file we want to run is in the current directory, we can use

```
dir
```

(or `ls`). If the file is not in there, we can use

```
cd
```

to change the current directory, moving within the directories of our system. The **cd** command is used by specifying the name of the directory we want to move in. So if the current directory is `C:\Users\Angelo` and we want to move into the inner Documents directory, we have to simply use the directory name:

```
C:\Users\Angelo> cd Documents
C:\Users\Angelo\Documents>
```

To go back into the outer directory, we use a special directory name that is `..` (two dots), as in the following:

```
C:\Users\Angelo\Documents> cd ..
C:\Users\Angelo>
```

For Linux and MacOS, the `cd` command has the same syntax.

Why is it important to know the current command line directory? Because when we want to run a python program, we must be sure that the file to run is in the current directory. Alternatively, we must specify to Python the full sequence of the directory, that is the absolute **file path**, that includes all the subdirectories.

Example: to run the `hello.py` file we created before, we could use the following commands (which are equivalent)

```
C:\Users\Angelo\Documents\Python Scripts> python hello.py
C:\Users\Angelo>python "C:\Users\Angelo\Documents\Python Scripts\hello.py"
```

WARNING!! Since the `hello.py` file we created before does NOT contain Python code, we will get an error... Moreover, observe that in the second version of the command we had to use double quotes (" ") because there is a whitespace character in the name of one of the directories, namely the `Python Scripts` one. Finally, in Linux or MacOS the two commands would be slightly different, since the directories are indicated with the slash (/), instead of the backslash (\), which is typical of Windows command lines.

How to use the Python interpreter

Python is an interpreted programming language (you will know what this means later in the course) and has an interactive command line exactly like the one we have to interact with the operating system: when we insert a command, Python will read it, will interpret it and, possibly, will execute it, showing the output of the command. One of the windows we opened, indeed, is exactly this command line and we can use it to start inserting some Python commands.

As you can see, the prompt is different w.r.t. the operating system, and it is composed of three symbols of major

```
>>>
```

As a starter, we can try some arithmetic operations as if we were using a calculator. Here there is a small list with some of the operators that Python can evaluate:

- + for the sum
- - for the difference
- * for product
- / for floating-point division
- // for the whole division
- % for the rest of the whole division
- ** for a power lift
- () brackets to change the priority of operators

Let us do some tests...

[illegible]

We continue discovering that Python provides the concept of **function**, which is a convenient way to define a series of instructions to be repeated and to give it a name. Moreover, Python provides a lot of functions that help you write useful programs. One of the functions that we will use very often is the **print** function, which is the function that prints something on the screen. When we want to use a function in Python we must always use round brackets. So we can ask Python to print on the screen the string *"Hello world!"* using round brackets and quotes to define a string in the following way:

```
>>> print("Hello world!")
Hello world!
>>>
```

We can also ask Python to print a string and the result of an operation, inserting the two elements separated by a comma:

```
>>> print ("Zero", 4+4-4-4)
Zero 0
>>>
```

How to run a Python program

Now, it is time to use our IDLE text editor (that should run in one of the open windows) to write our program. We will write a program with the same print command as above, to see what it happens. Thus, let us change the text of the hello.py file with this content:

```
print("Hello world!")
```

We already know that the above command essentially prints "Hello world!" on screen. We can observe that the characters we entered are colored differently. The IDLE editor helps distinguish the syntax of python: function names have one color, symbols have another and strings have another. We can save the file from the IDLE **File->Save menu**. Our program should be saved as C:/Users/Angelo/Documents/Python Scripts/hello.py (your path should be consistent with your username and home directory). Let us go in the operating system command line and let us make sure we are in the right directory—in case, remember to use the cd command to change the directory—and type dir (or ls) to make sure the file is there.

And now we can run the hello.py program by using the command **python hello.py** in the command line of the operating system:

```
C:\Users\Angelo\Documents\Python Scripts> python hello.py
>Hello world!
C:\Users\Angelo\Documents\Python Scripts>
```

Right after we press the Enter key, the program is executed, and, as expected, it simply prints the "Hello world!" string on the screen.

We can now create any other python program by simply selecting from our Python editor or Python interpreter the command **File->New File**.

Python challenge!

Try to write a program that prints on screen all the numbers from 0 to 20 using only four 4... Also numbers with decimals (i.e. 1.0, 2.0 etc.) and fractions (i.e. 0.4, 0.19) are good.

For example, zero is 4+4-4-4 (i.e. we are using four times the number four, using the right combination of arithmetic operators, in order to obtain the value 0). Can you write all the numbers up to 20?

You can start creating a new file with the following content:

```
# Four fours challenge
# Filename: fourfours.py
# Problem description: The four fours challenge!
from math import *
print("Zero is", 4+4-4-4)
```

and saving it with a name like fourfours.py. If you run the program as above described, you should obtain as an output the string "Zero is 0".

You can notice that in the program we inserted a line **from math import** : this is a special line for Python that allows us to use some special mathematical function like ***factorial** (to evaluate the factorial of a number) and **sqrt** (to evaluate the squared root of number): for example, factorial(5) evaluates $5!=5*4*3*2*1=120$ while sqrt(9) evaluates the square root of 9, namely 3:

```
>>> sqrt(9)
3
>>> factorial(5)
120
>>>
```